DOIS DIAS DE AULA · TOOLS, AGENTES E MCP

MCP: da Tool ao Protocolo

A evolução natural do uso de tools em LLMs até o Model Context Protocol, com exercício em FastMCP e LangChain.

Dia 1 · Evolução e arquitetura | Dia 2 · Mão na massa e boas práticas

AGENDA

Dois dias, uma história

Dia 1 · Evolução e arquitetura

Bloco	Duração
Recap: o loop de tool use	20 min
Function calling e frameworks	50 min
Intervalo	15 min
O problema N×M e o MCP	45 min
Arquitetura do MCP	70 min
Fechamento e demo	40 min

Dia 2 · Mão na massa

Bloco	Duração
LangChain em 5 conceitos	25 min
Exercício, Partes 1 e 2	35 min
Intervalo	15 min
FastMCP e Parte 3	60 min
Adapter e Parte 4	40 min
Segurança e decisão	35 min
Fechamento	10 min

OBJETIVOS

Ao final dos dois dias você será capaz de

1

Explicar

por que o MCP existe, reconstruindo a evolução de function calling até frameworks e protocolo.

2

Descrever

a arquitetura: host, client e server; tools, resources e prompts; stdio e HTTP.

3

Construir

um servidor MCP com FastMCP e testá-lo sem LLM, antes de ligar um agente.

4

Conectar

o servidor a um agente LangChain, ler o trace e avaliar riscos de segurança.

DIA 1 · PONTO DE PARTIDA

Recap: o loop de tool use

1	2	3	4	5
A aplicação envia a mensagem e os schemas das tools.	O modelo responde com um pedido: nome da tool e argumentos.	A aplicação executa a função de verdade.	O resultado volta ao modelo como mensagem de tool.	O modelo escreve a resposta final (ou pede outra tool).

Os passos 2 a 4 se repetem até o modelo ter o que precisa.

PERGUNTA-GUIA DOS DOIS DIAS

O modelo nunca executa nada.

Ele só pede. Quem executa é a aplicação. Então onde essa aplicação (e a tool) deve viver?

Quatro estágios de evolução

1

Function calling

Resolve: o modelo passa a pedir ações.

Deixa: schema, loop e formato por sua conta.

2

Frameworks

Resolve: schema automático e loop pronto.

Deixa: a tool presa ao código da aplicação.

3

Tools em escala

Resolve: muitos casos de uso.

Deixa: N×M integrações para manter.

4

MCP

Resolve: um contrato único entre apps e tools.

Deixa: segurança e governança como novo desafio.

ESTÁGIO 1

Function calling nativo

- Você descreve cada função em JSON Schema: nome, descrição e parâmetros.
- O modelo decide quando chamar e devolve JSON estruturado, não texto livre.
- Sua aplicação lê o JSON, executa e devolve o resultado.

O que mudou

O modelo deixou de só falar sobre o mundo e passou a agir sobre ele, mas sempre por meio da sua aplicação.

ESTÁGIO 1

O schema e o pedido do modelo

1. Você declara a tool

```
{
  "name": "consultar_preco",
  "description": "Preço de um prato",
  "parameters": {
    "type": "object",
    "properties": {
      "prato": {"type": "string"}
    },
    "required": ["prato"]
  }
}
```

2. O modelo responde com um pedido

```
{
  "name": "consultar_preco",
  "arguments": {
    "prato": "moqueca"
  }
}
```

Nada foi executado ainda. É só um pedido estruturado. O formato exato varia por provedor (simplificado aqui).

ESTÁGIO 1

O que você faz à mão

Schema

Escrever os parâmetros em JSON e mantê-los em sincronia com a função real.

Loop

Detectar o pedido, executar, reenviar o resultado, tratar erros e limitar iterações.

Formato por provedor

Cada API declara e devolve tools de um jeito. Trocar de modelo mexe no código.

Reuso quase zero

Cada projeto copia as mesmas definições e o mesmo loop.

ESTÁGIO 1

Mesma ideia, contratos diferentes

Provedor	Como declara a tool	Como o modelo pede
OpenAI	tools com função e parameters	tool_calls na resposta
Anthropic	tools com input_schema	bloco tool_use na resposta
Google Gemini	functionDeclarations	functionCall na resposta

Consequência: trocar de provedor, ou suportar vários, significa reescrever a camada de integração.

Exemplos ilustrativos; os nomes de campos podem mudar entre versões das APIs. Confira a documentação de cada provedor.

ESTÁGIO 2

O framework assume o trabalho

No estágio 1 você	No estágio 2 o framework
escreve o schema em JSON	gera o schema da assinatura e da docstring
escreve o loop de execução	entrega o loop pronto (o agente)
adapta o código a cada provedor	expõe uma interface única de modelo

Neste curso o exemplo é o **LangChain**. A ideia vale para outros frameworks de agentes.

ESTÁGIO 2

A mesma tool com @tool

```python
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def consultar_preco(prato: str) -> float:
  """Retorna o preço de um prato."""
  return PRECOS[prato.lower().strip()]

agent = create_agent(llm, [consultar_preco])
agent.invoke({"messages": msgs})
```

@tool

O schema vem da assinatura e da docstring.

create_agent

O loop de tool use já vem pronto.

Onde a função roda?

No mesmo processo da aplicação.

ESTÁGIO 2

O que o framework não resolve

Resolveu

- Schema automático
- Loop pronto
- Trocar de modelo com uma linha

Continua em aberto

- A tool vive dentro do processo da aplicação.
- Outra equipe, linguagem ou framework não reaproveita a tool.
- Não há descoberta de tools em tempo de execução.
- Permissões e versões ficam por conta de cada aplicação.

ESTÁGIO 3

O problema N×M

Aplicação	Banco	Agenda	Arquivos	CRM
Chatbot	integração	integração	integração	integração
IDE	integração	integração	integração	integração
Agente interno	integração	integração	integração	integração

Cada aplicação (N) precisa de uma integração própria com cada fonte (M).

12

integrações para escrever e manter

ESTÁGIO 3 → 4

Com um padrão, N×M vira N+M



3 + 4 = 7 peças, em vez de 12 integrações. Uma nova fonte custa 1, não 3.

ESTÁGIO 3 → 4

O que um padrão precisaria oferecer

1. **Descrição neutra:** a tool se descreve sem depender de provedor ou framework.
2. **Processo separado:** a tool roda fora da aplicação, até em outra máquina.
3. **Descoberta:** o cliente pergunta quais tools existem, em vez de tê-las no código.
4. **Transporte padrão:** uma forma única de conversar, local ou remota.
5. **Controle de acesso:** autenticação e permissões previstas no contrato.

Esse problema já foi resolvido uma vez, em outra área.

ESTÁGIO 3 → 4

Já vimos esse filme

Mundo	O problema	O padrão
Editores e linguagens	Cada editor reimplementa o suporte a cada linguagem.	Language Server Protocol (LSP)
Aplicações de IA e fontes de dados	Cada aplicação reimplementa a integração com cada fonte.	Model Context Protocol (MCP)

O MCP se inspira no LSP: o servidor é escrito uma vez e serve qualquer cliente compatível.

ESTÁGIO 4

Model Context Protocol

Um protocolo aberto que padroniza como aplicações de IA se conectam a ferramentas e dados.

Mensagens

Trocadas em JSON-RPC 2.0, entre cliente e servidor.

Origem

Lançado pela Anthropic em 2024, com especificação pública e adotado por vários fornecedores.

Efeito

Um servidor serve qualquer cliente compatível, em qualquer linguagem.

SÍNTESE DA EVOLUÇÃO

Três formas de dar tools ao modelo

	Function calling	Framework	MCP
Quem descreve	Você, em JSON	Assinatura e docstring	O servidor, em contrato padrão
Onde roda	Na aplicação	No processo da aplicação	Em processo ou serviço separado
Reuso	Copiar e colar	Dentro do framework	Qualquer cliente MCP
Descoberta	Fixa no código	Fixa no código	Em tempo de execução

DEMO AO VIVO · 15 MIN

As mesmas tools, em dois lugares

1. Estágio 2: rodar a tool local e ler o trace.
2. Listar as tools do servidor, sem LLM.
3. Estágio 4: rodar o agente com tools vindas do servidor.

Pergunta: o que mudou no código do agente?

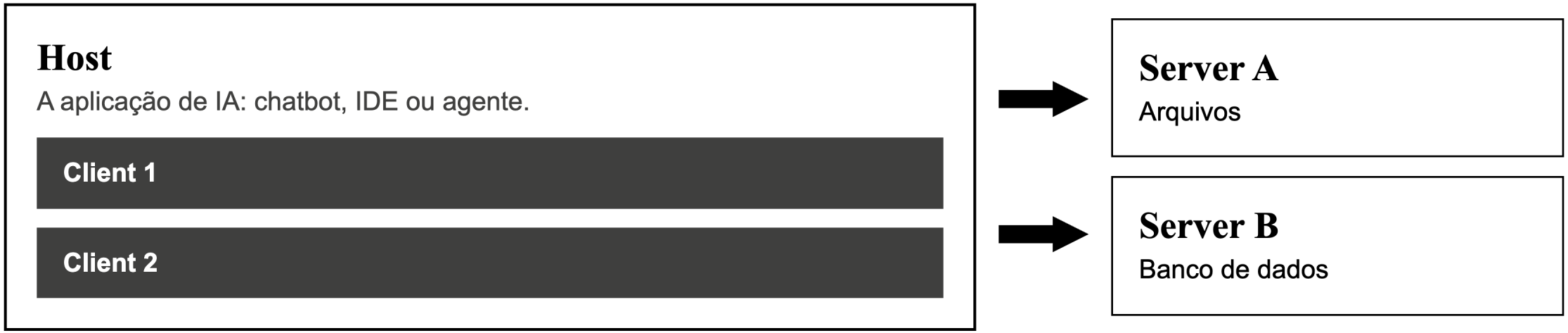
```
# 1. tool local (estágio 2)
python parte2_tool_local.py

# 2. tools do servidor, sem LLM
fastmcp list \
  gabarito_servidor_restaurante.py

# 3. agente + MCP (estágio 4)
python parte4_agente_mcp.py
```

DIA 1 · ARQUITETURA

Host, client e server



Cada client mantém uma conexão 1:1 com um servidor. O host cria um client por servidor.

DIA 1 · ARQUITETURA

Três camadas

Primitivas	**O que servidor e cliente oferecem.** Tools, resources e prompts (servidor); sampling, elicitation e roots (cliente).
Protocolo	**O formato das mensagens.** JSON-RPC 2.0: requests, responses e notifications.
Transporte	**Como os bytes viajam.** stdio (processo local) ou Streamable HTTP (serviço remoto).

DIA 1 · ARQUITETURA

Ciclo de vida de uma conexão

1

Initialize

O client envia a versão do protocolo e suas capacidades.

2

Resposta

O servidor devolve as suas capacidades: tools, resources, prompts.

3

Operação

O client lista e usa o que existe: tools/list, tools/call, resources/read.

4

Encerramento

A conexão fecha. No stdio, o processo do servidor termina.

Fluxo clássico de handshake. Revisões recentes da especificação mudaram partes dele; veja o slide sobre a spec 2026-07-28.

DIA 1 · ARQUITETURA

O que viaja no fio: JSON-RPC

Request: tools/call

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "consultar_preco",
    "arguments": {"prato": "moqueca"}
  }
}
```

Response (mesmo id)

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      {"type": "text", "text": "58.0"}
    ]
  }
}
```

Simplificado. Antes disso, **tools/list** devolveu o nome, a descrição e o schema de entrada de cada tool: é a descoberta em tempo de execução.

Tools, resources e prompts

Tools

Controle: o modelo

Ações e consultas que o modelo decide chamar.

Exemplo: calcular_conta

Resources

Controle: a aplicação

Dados de leitura, identificados por URI, que a aplicação escolhe anexar ao contexto.

Exemplo: restaurante://horario

Prompts

Controle: o usuário

Modelos de interação reutilizáveis, que o usuário escolhe e preenche.

Exemplo: sugerir_para_grupo

O servidor também pode pedir

Sampling

O servidor pede ao client uma resposta do LLM do host, sem ter chave de API própria.

Elicitation

O servidor pede ao usuário uma informação que faltou, no meio de uma operação.

Roots

O client informa ao servidor em quais pastas ou locais ele pode trabalhar.

Capacidades opcionais: nem todo cliente as suporta. No exercício usaremos só as primitivas do servidor.

DIA 1 · TRANSPORTES

stdio ou Streamable HTTP

	stdio	Streamable HTTP
Servidor	Subprocesso na mesma máquina	Serviço com uma URL (local ou remoto)
Conversa	Pela entrada e saída padrão	Por requisições HTTP a um endpoint único
Uso típico	Ferramentas locais e desenvolvimento	Serviços compartilhados e multiusuário
Cuidado	Nunca imprimir no stdout; usar logs	Autenticação, TLS e controle de origem

O transporte antigo HTTP+SSE está depreciado; novos servidores usam Streamable HTTP.

DIA 1 · ATUALIZAÇÃO

O que mudou na spec 2026-07-28

No protocolo

- Cada requisição leva a versão do protocolo e as capacidades do client em _meta.
- Tasks experimentais saem do núcleo e viram uma extensão oficial.
- Novo campo extensions nas capacidades.

No transporte HTTP

- Sem sessões no nível do protocolo e sem o header Mcp-Session-Id.
- Sem retomada de stream SSE (Last-Event-ID).
- Headers padrão Mcp-Method e Mcp-Name nos POSTs.
- HTTP+SSE passa a Deprecated.

Na prática: confira qual revisão da spec a sua biblioteca implementa antes de depender de um comportamento.

DIA 1 · FECHAMENTO

O que levar do Dia 1

1. O modelo só pede; quem executa é a aplicação.
2. Cada estágio herda o problema do anterior: function calling, frameworks, N×M, MCP.
3. MCP é um protocolo cliente-servidor em JSON-RPC, com tools, resources e prompts, sobre stdio ou HTTP.

Para o Dia 2

- Criar o ambiente virtual e instalar o requirements.txt.
- Definir MODELO e a chave de API do provedor.
- Rodar a Parte 1 do exercício.

DIA 2 · MÃO NA MASSA

Mapa do exercício

Parte	O que fazemos	Tempo	Tipo
1	Primeiro contato com LangChain	15 min	Núcleo
2	Tool local com @tool e create_agent	20 min	Núcleo
3	Escrever um servidor MCP com FastMCP	40 min	Núcleo
4	Ligar o agente ao servidor via adapter	25 min	Núcleo
5A	Dois servidores, stdio e HTTP	30 min	Desafio
5B	Resources e prompts no LangChain	20 min	Desafio

Todo o material está na pasta do exercício, com README, gabarito e problemas comuns.

DIA 2 · LANGCHAIN PARA INICIANTES

LangChain em 5 conceitos

Conceito	Para que serve	Como aparece no código
Modelo	Conversar com o LLM de qualquer provedor	init_chat_model("provedor:modelo")
Mensagens	Histórico: system, user, assistant, tool	{"role": "user", "content": ...}
Tool	Função que o modelo pode pedir	@tool
Agente	Loop que chama modelo e tools até terminar	create_agent(modelo, tools)
Trace	A conversa completa, passo a passo	resultado["messages"]

Modelo, mensagens e invoke

```python
from langchain.chat_models import init_chat_model

llm = init_chat_model("openai:gpt-4.1")

resposta = llm.invoke("O que é tambaqui?")
print(type(resposta).__name__)
print(resposta.content)
```

"provedor:modelo"

Troque pelo provedor e modelo da sua conta.

invoke

Envia as mensagens e devolve uma AIMessage; o texto está em .content.

Papéis

system, user, assistant e tool formam o histórico.

Tool local e primeiro agente

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def consultar_preco(prato: str) -> float:
  """Retorna o preço de um prato."""
  return PRECOS[prato.lower().strip()]

agent = create_agent(llm, [consultar_preco])
msgs = [{"role": "user",
    "content": "Preço da moqueca?"}]
resultado = agent.invoke({"messages": msgs})
```

O loop do agente

1. Chama o modelo com as mensagens.
2. Se ele pediu tools, executa cada uma.
3. Anexa os resultados e chama o modelo de novo.
4. Para quando o modelo responde sem pedir tools.

Lendo um trace

```
[HumanMessage] Quanto custa a moqueca mais o tacacá?
[AIMessage] chamou tool: consultar_preco({'prato': 'moqueca'})
[ToolMessage] 58.0
[AIMessage] chamou tool: consultar_preco({'prato': 'tacacá'})
[ToolMessage] 24.0
[AIMessage] A moqueca e o tacacá somam R$ 82,00.
```

HumanMessage	**AIMessage + tool_calls**	**ToolMessage**	**AIMessage final**
Você.	O modelo pediu.	O código executou.	O modelo respondeu.

Trace ilustrativo, no formato do helper mostrar_trace do exercício. Alguns modelos pedem as duas tools de uma vez.

EXERCÍCIO · 35 MIN

Partes 1 e 2: LangChain e a tool local

- Rode parte1_langchain_basico.py e confirme a AIMessage.
- Rode parte2_tool_local.py e leia o trace.
- Desafio: pergunte por um prato que não existe. O que acontece com o KeyError?

Anatomia de um servidor FastMCP

```python
from fastmcp import FastMCP

mcp = FastMCP("restaurante")

@mcp.tool
def consultar_preco(prato: str) -> float:
  """Retorna o preço de um prato."""
  return _preco(prato)

if __name__ == "__main__":
  mcp.run()
```

FastMCP("nome")

Cria o servidor e o identifica.

@mcp.tool

Registra a função como tool do servidor.

mcp.run()

Inicia o servidor; o transporte padrão é stdio.

Assinatura e docstring viram contrato

O que você escreve

```python
@mcp.tool
def consultar_preco(prato: str) -> float:
  """Retorna o preço de um
  prato pelo nome exato."""
  return _preco(prato)
```

O que o cliente descobre (tools/list)

```
{
  "name": "consultar_preco",
  "description": "Retorna o preço…",
  "inputSchema": {
    "type": "object",
    "properties": {
      "prato": {"type": "string"}
    },
    "required": ["prato"]
  }
}
```

Tipos e docstring são a interface com o modelo. O FastMCP gera o schema; você não escreve JSON.

Resource e prompt

```python
@mcp.resource("restaurante://horario")
def horario() -> str:
  """Horário de funcionamento."""
  return "Terça a domingo, 11h às 22h."

@mcp.prompt
def sugerir_para_grupo(pessoas: int) -> str:
  """Pede uma sugestão de pedido."""
  return f"Sugira um pedido para {pessoas}."
```

Resource

Dado identificado por URI. A aplicação decide anexá-lo ao contexto.

Prompt

Modelo parametrizado. O usuário escolhe e preenche.

Versão reduzida do prompt do exercício, que também recebe restricao.

Teste o servidor antes do agente

```
python parte3_teste_cliente.py

fastmcp list parte3_servidor_restaurante.py

fastmcp call parte3_servidor_restaurante.py \
    consultar_preco prato=moqueca

fastmcp dev inspector parte3_servidor_restaurante.py
```

Client em Python
Roda os testes do roteiro.

fastmcp list e call
Lista e chama pela linha de comando.

Inspector (opcional)
Interface no navegador; requer Node.

Se o servidor não funciona sozinho, o agente não vai funcionar. Os comandos fastmcp exigem o ambiente virtual ativado.

EXERCÍCIO · 40 MIN

Parte 3: escreva o seu servidor MCP

1. consultar_preco, com erro que ajude o modelo a se corrigir.
2. calcular_conta, com subtotal, gorjeta e total.
3. Resource restaurante://horario.
4. Prompt sugerir_para_grupo.

Teste cada TODO sem LLM antes de seguir. Gabarito: gabarito_servidor_restaurante.py.

O adapter: de MCP para LangChain

```
client = MultiServerMCPClient({
  "restaurante": {
    "transport": "stdio",
    "command": sys.executable,
    "args": ["servidor.py"],
  }
})

tools = await client.get_tools()
agent = create_agent(llm, tools)
```

get_tools()
Descobre as tools no servidor e as converte em tools LangChain.

create_agent
Igual à Parte 2. Só muda a origem das tools.

stdio
O client inicia o servidor como subprocesso.

DIA 2 · DESAFIO 5A

Dois servidores, dois transportes

```
client = MultiServerMCPClient({
  "restaurante": {
    "transport": "stdio",
    "command": sys.executable,
    "args": ["restaurante.py"],
  },
  "reservas": {
    "transport": "streamable_http",
    "url": "http://127.0.0.1:8000/mcp",
  },
})
```

Um agente, dois servidores

As tools dos dois chegam juntas ao modelo.

No servidor HTTP

`mcp.run(transport="http")`

O que o agente vê

Nada muda: o transporte é detalhe do client.

DIA 2 · DESAFIO 5B

Resources e prompts não viram tools

```
async with client.session("restaurante") as s:
  blobs = await load_mcp_resources(
    s, uris=["restaurante://horario"])
  msgs = await load_mcp_prompt(
    s, "sugerir_para_grupo",
    arguments={"pessoas": "4"})
```

Resource

Contexto que a sua aplicação escolhe injetar.

Prompt

Mensagens prontas, geradas pelo servidor.

Atenção

Argumentos de prompt são sempre strings: "4", não 4.

EXERCÍCIO · 25 MIN + DESAFIOS

Parte 4: ligue o agente ao seu servidor

- Aponte parte4_agente_mcp.py para o seu servidor.
- Leia o trace: que tools o modelo escolheu, e em que ordem?
- Compare com a Parte 2: o que mudou no código do agente?

Desafios opcionais: 5A (dois servidores, stdio e HTTP) e 5B (resources e prompts).

DIA 2 · DEPURAÇÃO

Armadilhas encontradas no exercício

Sintoma	Causa provável	Correção
Connection closed	O servidor falhou ao iniciar ou escreveu no stdout.	Rode o servidor direto; use logs, não print.
ImportError em RequestContext	Pacote mcp 2.x com o adapter testado (0.3.1).	Ambiente limpo com o requirements.txt.
Input should be a valid string	Argumento de prompt que não é string.	Passe "4", não 4.
Connection refused	O servidor HTTP não está rodando.	Suba-o em outro terminal.

DIA 2 · SEGURANÇA

O servidor entra no seu agente

Injeção por resultado

Texto devolvido por uma tool pode trazer instruções que o modelo obedece.

Descrição maliciosa

Nome e descrição da tool entram no prompt. Um servidor hostil pode manipulá-los.

Permissão em excesso

Uma tool com acesso amplo transforma um erro do modelo em dano real.

Combinação perigosa

Dados privados, conteúdo não confiável e saída para a rede, juntos, permitem vazar informação.

DIA 2 · SEGURANÇA

Boas práticas de segurança

Prática	Como aplicar
Mínimo privilégio	Credenciais e acessos restritos ao que a tool realmente precisa.
Confirmação humana	Ações destrutivas pedem aprovação; anotações de tool sinalizam leitura ou destruição.
Servidores confiáveis	Lista permitida de servidores; revisão de código e de descrições.
Autenticação em HTTP	A especificação prevê autorização baseada em OAuth 2.1.
Saída não confiável	Trate o retorno de tools como dado, não como instrução; registre as chamadas.

DIA 2 · DESIGN DE TOOLS

Projetando boas tools

Poucas tools

Cada tool a mais é mais uma escolha para o modelo errar.

Nome e docstring

Verbo claro e descrição objetiva: é o manual de uso do modelo.

Parâmetros simples

Tipos primitivos e valores fixos, em vez de objetos livres.

Erros acionáveis

Diga o que falhou e o que tentar: "use listar_pratos()".

Retorno enxuto

Devolva só o que o modelo precisa, não a resposta inteira da API.

Tarefas, não endpoints

Modele o que o usuário quer fazer, não cada rota da API.

DIA 2 · DECISÃO

Tool local ou servidor MCP?

Pergunta	Tool local (@tool)	Servidor MCP
Quem usa?	Só esta aplicação	Várias aplicações ou equipes
Quem mantém?	A equipe do agente	Outra equipe ou fornecedor
Isolamento?	Não é necessário	Credenciais e permissões à parte
Complexidade?	Protótipo, lógica simples	Mais peças: processo e transporte

Atividade em dupla (10 min): local ou MCP? Justifique.

1. Uma função que formata datas para o seu chatbot.
2. Acesso ao CRM para o chatbot, a IDE e um agente de vendas.
3. Consulta ao banco de dados mantida pela equipe de dados.
4. Função que soma números, em um protótipo.

FECHAMENTO

O ecossistema ao redor do protocolo

Hosts

Assistentes de IA, IDEs e agentes que se conectam a servidores MCP.

Servidores

Arquivos, código, bancos de dados e serviços, de fornecedores e da comunidade.

SDKs e frameworks

Bibliotecas em várias linguagens, como o FastMCP, e adapters para frameworks como o LangChain.

Servidor da comunidade é código de terceiros. Aplique os cuidados de segurança do Dia 2.

FECHAMENTO

A evolução em quatro passos

Function calling
O modelo pede, o JSON descreve.

→

Frameworks
@tool e o loop prontos.

→

Tools em escala
N×M integrações.

→

MCP
Contrato único: N+M.

Onde a tool vive?

No MCP, em um servidor separado, descrita por um contrato padrão e descoberta em tempo de execução. O modelo continua só pedindo.

FECHAMENTO

Para continuar

Tema	Onde
Especificação e guias do MCP	modelcontextprotocol.io
LangChain com MCP	docs.langchain.com/oss/python/langchain/mcp
Adapter LangChain e MCP	github.com/langchain-ai/langchain-mcp-adapters
FastMCP	pypi.org/project/fastmcp
Material do curso	Pasta do exercício: README, gabarito e servidores